

Design and Implementation of the CoWorkHub Database System

Mary-Jane Masagbor

1.1 INTRODUCTION AND SCENARIO

CoWorkHub: Co-working space membership

CoWorkHub is a co-working space that offers desk areas and bookable meeting rooms. Customers register as members on a subscription plan and can reserve rooms for specific time slots or durations. Some bookings require payments depending on room type and plan entitlements. For safety and governance, the building uses door scanners that record access attempts by members. Staff need the database to manage members and plans, control room inventory and bookings, track booking payments, and audit access events.

1.2 BUSINESS RULES AND INFORMATION NEEDS.

BUSINESS RULES

- Each member must be linked to exactly one plan.
- A plan can be linked to many members.
- Each booking is created by exactly one member.
- Each booking reserves exactly one room.
- A room can have many bookings over time.
- Bookings for the same room must not overlap in time.
- Booking end time must be after start time.
- Each booking can have at most one payment record.
- Each payment belongs to exactly one booking.
- Member email addresses must be unique.
- Access logs must record the member, time, location, and result (granted or denied).
- An access log may optionally reference a booking (if entry is related to a specific meeting).

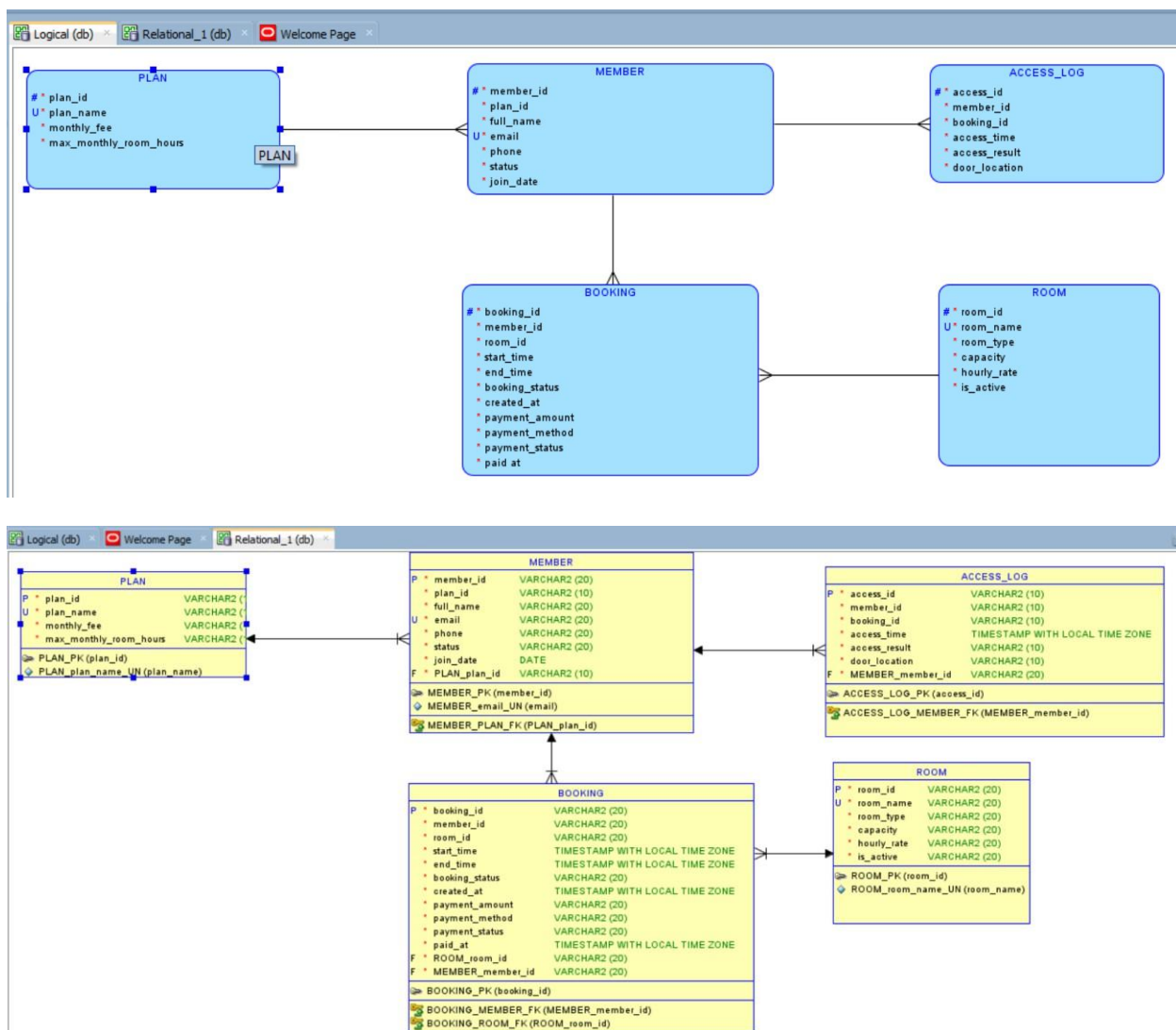
EXAMPLE SCENARIOS

- A new member signs up, chooses a plan, and makes a room booking.
- A member pays for a booking, and the finance team confirms payment status.
- A member attempts to enter the building and the system logs whether access was granted.
- Staff analyst which rooms generate the highest revenue.

INFORMATION NEEDS

- What bookings does each member have and which rooms would they be using?
- What revenue has each room generated within a period?
- Which bookings are unpaid or failed payments?
- Which members have repeated denied access attempts or out of hours access events?
- How many members are on each plan and which plan is the most profitable?

1.3 ER MODEL



The ER model for the CoWorkHub system represents the key business processes involved in managing memberships, room bookings, and building access monitoring. The design contains five main entities: PLAN, MEMBER, ROOM, BOOKING, and ACCESS_LOG. Each entity represents a core concept within the co-working environment.

The PLAN entity stores membership subscription types offered by the co-working space, including pricing and monthly booking allowances. Each plan can cover multiple members, but each member must belong to exactly one plan. This relationship ensures that the benefits and pricing structure are consistently applied to members.

The MEMBER entity represents individuals registered with the co-working space. Members store personal and contact information such as name, email, and phone number. Each member is associated with one plan, and a unique email constraint ensures that duplicate accounts cannot exist. Members are responsible for creating bookings and generating access events within the system.

The ROOM entity represents physical spaces available for booking, such as meeting rooms, desks, or private pods. Each room has a capacity, hourly rate, and operational status. A room may be reserved many times over different time periods, but each booking refers to exactly one room.

The BOOKING entity records reservations made by members for specific rooms. Each booking is associated with one member and one room, establishing the relationship between users and the resources they reserve. The booking entity also includes payment-related attributes such as payment amount, method, and payment status, allowing financial tracking without introducing a separate payment entity. The start and end time attributes allow the system to enforce scheduling constraints and prevent invalid bookings.

The ACCESS_LOG entity records entry attempts into the building through door scanners. Each access event belongs to a specific member and records the time, location, and whether the access attempt was granted or denied. In some cases, an access event may also reference a booking if the entry is related to a scheduled meeting. This relationship supports auditing and security monitoring.

Overall, the relationships between entities reflect the operational workflow of the co-working environment. Members subscribe to plans, create bookings for rooms, and generate access records when entering the building. The model supports operational management, financial tracking, and security auditing while maintaining clear entity relationships and data integrity.

1.3 NORMALISATION

Unnormalised Form

At first, the system information could be stored in one large table that mixes membership, plan, room, booking, payment, and access event details in the same place.

UNF structure

CO_WORKING_RECORD (UNF columns)

- MemberName
- MemberEmail
- PlanName
- PlanMonthlyFee
- PlanMaxMonthlyRoomHours
- RoomName
- RoomType
- RoomCapacity
- RoomHourlyRate
- BookingStartTime
- BookingEndTime
- BookingStatus
- PaymentAmount
- PaymentMethod
- PaymentStatus
- PaidAt
- AccessTime (repeating)
- DoorLocation (repeating)
- AccessResult (repeating)

Why UNF is a problem

- Plan details repeat for every member and booking.
- Room details repeat for every booking.
- A member can have many access events, which creates repeating groups in one row.
- Updates become inconsistent (for example, changing a room hourly rate would require changing it in many rows).

First Normal Form (1NF)

Rule of 1NF: attributes must be atomic (single values) and there must be no repeating groups.

In UNF, access information repeats (a member can have many access events), so it violates 1NF. To reach 1NF, access events must be stored as separate rows (one access event per row). The repeating access-related attributes are separated so each access event is stored as its own row in a log.

Changes in 1NF

- AccessTime, DoorLocation and AccessResult are removed from the single table repeating group and become rows in an ACCESS_LOG structure.
- All remaining columns are treated as atomic values (one value per cell).

At this point, duplication still exists because plan and room details still repeat for many bookings.

Second Normal Form (2NF)

Rule of 2NF: the design must be in 1NF and have no partial dependency on a composite key. In a single wide booking record, attributes like PlanMonthlyFee depend only on PlanName, not on the entire booking identifier. Similarly, RoomHourlyRate depends only on RoomName. These partial dependencies are removed by splitting data into separate tables.

Identified functional dependencies

- PlanName → PlanMonthlyFee, PlanMaxMonthlyRoomHours
- MemberEmail → MemberName (and other member details)
- RoomName → RoomType, RoomCapacity, RoomHourlyRate

- (Member, Room, StartTime, EndTime) → BookingStatus (booking event details)

2NF outcome

To remove partial dependencies, the design is decomposed into the following

Table	Key attributes
PLAN	plan_id (PK), plan_name (UQ), monthly_fee, max_monthly_room_hours
MEMBER	member_id (PK), plan_id (FK), full_name, email (UQ), phone, status, join_date
ROOM	room_id (PK), room_name (UQ), room_type, capacity, hourly_rate, is_active
BOOKING	booking_id (PK), member_id (FK), room_id (FK), start_time, end_time, booking_status, created_at, payment_amount, payment_method, payment_status, paid_at
ACCESS_LOG	access_id (PK), member_id (FK), booking_id (FK, nullable), access_time, access_result, door_location

Third Normal Form (3NF)

Rule of 3NF: the design must be in 2NF and contain no transitive dependencies. A transitive dependency occurs when a non-key attribute depends on another non-key attribute rather than the primary key.

Examples of transitive dependency avoided

- MEMBER should not store monthly_fee because monthly_fee depends on plan_id (stored in PLAN).
- BOOKING should not store hourly_rate because hourly_rate depends on room_id (stored in ROOM).

In the final schema, each non-key attribute depends only on its table's primary key. Therefore, the schema satisfies 3NF.

Final 3NF schema

The following tables form the final 3NF design used for implementation:

Table	Primary Key	Foreign Keys
PLAN	plan_id	—
MEMBER	member_id	plan_id → PLAN(plan_id)
ROOM	room_id	—
BOOKING	booking_id	member_id → MEMBER(member_id); room_id → ROOM(room_id)

ACCESS_LOG	access_id	member_id MEMBER(member_id); booking_id BOOKING(booking_id) (nullable)	→ →
------------	-----------	--	--------

PLAN

plan_id	plan_name	monthly_fee	max_monthly_room_hours
1	Starter	50.00	5.0
2	Pro	120.00	15.0

MEMBER

member_id	plan_id	full_name	email	phone	status	join_date
1	1	Ada Okafor	ada.okafor@cohub.com	07000000001	ACTIVE	2026-02-01
2	2	Ben Hughes	ben.hughes@cohub.com	07000000002	ACTIVE	2026-02-10

ROOM

room_id	room_name	room_type	capacity	hourly_rate	is_active
1	Room A	MEETING	6	20.00	Y
2	Phone Pod 1	POD	1	8.00	Y

BOOKING (core fields)

booking_id	member_id	room_id	start_time	end_time	booking_status	created_at
1	1	1	2026-03-02 10:00	2026-03-02 12:00	CONFIRMED	2026-03-01 18:00
2	2	2	2026-03-02 09:00	2026-03-02 10:00	COMPLETED	2026-03-02 08:30

BOOKING (payment fields stored in BOOKING)

booking_id	payment_amount	payment_method	payment_status	paid_at
1	40.00	CARD	PAID	2026-03-01 18:30
2	8.00	BANK	PAID	2026-03-02 08:55

ACCESS_LOG

access_id	member_id	booking_id	access_time	access_result	door_location
1	1	1	2026-03-02 09:50	GRANTED	Main Entrance
2	2	2	2026-03-02 08:40	DENIED	Main Entrance
3	2	NULL	2026-03-03 23:10	DENIED	Side Door

By decomposing the initial UNF structure into separate tables for plans, members, rooms, bookings (including payment attributes), and access events, the final schema removes repeating groups, partial dependencies, and transitive dependencies. This produces a maintainable 3NF design that reduces redundancy and supports data integrity through primary and foreign keys.

PART II

II.1 IMPLEMENTATION

INPUT DDL

```
CREATE TABLE ACCESS_LOG
(
    access_id          VARCHAR2 (10) NOT NULL ,
    member_id          VARCHAR2 (10) NOT NULL ,
    booking_id          VARCHAR2 (10) NOT NULL ,
    access_time         TIMESTAMP WITH LOCAL TIME ZONE NOT NULL ,
    access_result       VARCHAR2 (10) NOT NULL ,
    door_location       VARCHAR2 (10) NOT NULL ,
    MEMBER_member_id    VARCHAR2 (20) NOT NULL
)
;

ALTER TABLE ACCESS_LOG
    ADD CONSTRAINT ACCESS_LOG_PK PRIMARY KEY ( access_id ) ;

CREATE TABLE BOOKING
(
    booking_id          VARCHAR2 (20) NOT NULL ,
    member_id           VARCHAR2 (20) NOT NULL ,
    room_id             VARCHAR2 (20) NOT NULL ,
    start_time          TIMESTAMP WITH LOCAL TIME ZONE NOT NULL ,
    end_time            TIMESTAMP WITH LOCAL TIME ZONE NOT NULL ,
    booking_status      VARCHAR2 (20) NOT NULL ,
    created_at          TIMESTAMP WITH LOCAL TIME ZONE NOT NULL ,
    payment_amount      VARCHAR2 (20) NOT NULL ,
    payment method     VARCHAR2 (20) NOT NULL ,
    payment_status      VARCHAR2 (20) NOT NULL ,
    paid_at             TIMESTAMP WITH LOCAL TIME ZONE NOT NULL ,
    ROOM_room_id        VARCHAR2 (20) NOT NULL ,
    MEMBER_member_id    VARCHAR2 (20) NOT NULL
)
;

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_PK PRIMARY KEY ( booking_id ) ;

CREATE TABLE MEMBER
(
    member_id           VARCHAR2 (20) NOT NULL ,
    plan_id             VARCHAR2 (10) NOT NULL ,
    full_name           VARCHAR2 (20) NOT NULL ,
    email               VARCHAR2 (20) NOT NULL ,
    phone               VARCHAR2 (20) NOT NULL ,
    status              VARCHAR2 (20) NOT NULL ,
    join_date           DATE NOT NULL ,
    PLAN_plan_id        VARCHAR2 (10) NOT NULL
)
;

ALTER TABLE MEMBER
    ADD CONSTRAINT MEMBER_PK PRIMARY KEY ( member_id ) ;

ALTER TABLE MEMBER
    ADD CONSTRAINT MEMBER_email_UN UNIQUE ( email ) ;

CREATE TABLE PLAN
(
    plan_id             VARCHAR2 (10) NOT NULL ,
    plan_name           VARCHAR2 (10) NOT NULL ,
    monthly_fee         VARCHAR2 (12) NOT NULL ,
    max_monthly_room_hours VARCHAR2 (12) NOT NULL
)
;

ALTER TABLE PLAN
    ADD CONSTRAINT PLAN_PK PRIMARY KEY ( plan_id ) ;

ALTER TABLE PLAN
    ADD CONSTRAINT PLAN_plan_name_UN UNIQUE ( plan_name ) ;

CREATE TABLE ROOM
(
    room_id             VARCHAR2 (20) NOT NULL ,
    room_name           VARCHAR2 (20) NOT NULL ,
    room_type           VARCHAR2 (20) NOT NULL ,
    capacity            VARCHAR2 (20) NOT NULL ,
    hourly_rate        VARCHAR2 (20) NOT NULL ,
    is_active           VARCHAR2 (20) NOT NULL
)
;

ALTER TABLE ROOM
    ADD CONSTRAINT ROOM_PK PRIMARY KEY ( room_id ) ;

ALTER TABLE ROOM
    ADD CONSTRAINT ROOM_room_name_UN UNIQUE ( room_name ) ;

ALTER TABLE ACCESS_LOG
    ADD CONSTRAINT ACCESS_LOG_MEMBER_FK FOREIGN KEY
(
    MEMBER_member_id
)
REFERENCES MEMBER
(
    member_id
)
;

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_MEMBER_FK FOREIGN KEY
(
    MEMBER_member_id
)
REFERENCES MEMBER
(
    member_id
)
;

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_ROOM_FK FOREIGN KEY
(
    ROOM_room_id
)
REFERENCES ROOM
(
    room_id
)
;

ALTER TABLE MEMBER
    ADD CONSTRAINT MEMBER_PLAN_FK FOREIGN KEY
(
    PLAN_plan_id
)
REFERENCES PLAN
(
    plan_id
)
;
```

DROP TABLES

```
BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE access_log';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE payment';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE booking';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE room';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE member';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

```
BEGIN
| EXECUTE IMMEDIATE 'DROP TABLE plan';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

CREATE TABLES

```
CREATE TABLE PLAN
(
  plan_id          VARCHAR2(10) NOT NULL,
  plan_name        VARCHAR2(10) NOT NULL,
  monthly_fee      VARCHAR2(12) NOT NULL,
  max_monthly_room_hours VARCHAR2(12) NOT NULL
);

ALTER TABLE PLAN
  ADD CONSTRAINT PLAN_PK PRIMARY KEY (plan_id);

ALTER TABLE PLAN
  ADD CONSTRAINT PLAN_plan_name_UN UNIQUE (plan_name);

CREATE TABLE MEMBER
(
  member_id        VARCHAR2(20) NOT NULL,
  plan_id          VARCHAR2(10) NOT NULL,
  full_name        VARCHAR2(20) NOT NULL,
  email            VARCHAR2(20) NOT NULL,
  phone            VARCHAR2(20) NOT NULL,
  status           VARCHAR2(20) NOT NULL,
  join_date        DATE NOT NULL,
  PLAN_plan_id     VARCHAR2(10) NOT NULL
);
```

```
ALTER TABLE MEMBER
  ADD CONSTRAINT MEMBER_PK PRIMARY KEY (member_id);

ALTER TABLE MEMBER
  ADD CONSTRAINT MEMBER_email_UN UNIQUE (email);

ALTER TABLE MEMBER
  ADD CONSTRAINT MEMBER_PLAN_FK FOREIGN KEY (PLAN_plan_id)
  REFERENCES PLAN (plan_id);

CREATE TABLE ROOM
(
  room_id          VARCHAR2(20) NOT NULL,
  room_name        VARCHAR2(20) NOT NULL,
  room_type        VARCHAR2(20) NOT NULL,
  capacity         VARCHAR2(20) NOT NULL,
  hourly_rate      VARCHAR2(20) NOT NULL,
  is_active        VARCHAR2(20) NOT NULL
);

ALTER TABLE ROOM
  ADD CONSTRAINT ROOM_PK PRIMARY KEY (room_id);

ALTER TABLE ROOM
  ADD CONSTRAINT ROOM_room_name_UN UNIQUE (room_name);
```

```

CREATE TABLE BOOKING
(
    booking_id          VARCHAR2(20) NOT NULL,
    member_id           VARCHAR2(20) NOT NULL,
    room_id             VARCHAR2(20) NOT NULL,
    start_time          TIMESTAMP WITH LOCAL TIME ZONE NOT NULL,
    end_time            TIMESTAMP WITH LOCAL TIME ZONE NOT NULL,
    booking_status       VARCHAR2(20) NOT NULL,
    created_at          TIMESTAMP WITH LOCAL TIME ZONE NOT NULL,
    payment_amount       VARCHAR2(20) NOT NULL,
    payment_method       VARCHAR2(20) NOT NULL,
    payment_status       VARCHAR2(20) NOT NULL,
    paid_at             TIMESTAMP WITH LOCAL TIME ZONE NOT NULL,
    ROOM_room_id        VARCHAR2(20) NOT NULL,
    MEMBER_member_id    VARCHAR2(20) NOT NULL
);

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_PK PRIMARY KEY (booking_id);

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_MEMBER_FK FOREIGN KEY (MEMBER_member_id)
    REFERENCES MEMBER (member_id);

ALTER TABLE BOOKING
    ADD CONSTRAINT BOOKING_ROOM_FK FOREIGN KEY (ROOM_room_id)
    REFERENCES ROOM (room_id);

CREATE TABLE ACCESS_LOG
(
    access_id           VARCHAR2(10) NOT NULL,
    member_id           VARCHAR2(10) NOT NULL,
    booking_id          VARCHAR2(10) NOT NULL,
    access_time          TIMESTAMP WITH LOCAL TIME ZONE NOT NULL,
    access_result        VARCHAR2(10) NOT NULL,
    door_location        VARCHAR2(10) NOT NULL,
    MEMBER_member_id    VARCHAR2(20) NOT NULL
);

ALTER TABLE ACCESS_LOG
    ADD CONSTRAINT ACCESS_LOG_PK PRIMARY KEY (access_id);

ALTER TABLE ACCESS_LOG
    ADD CONSTRAINT ACCESS_LOG_MEMBER_FK FOREIGN KEY (MEMBER_member_id)
    REFERENCES MEMBER (member_id);

```

The tables have been created using the CREATE TABLE statements based on the ER Model developed. The schema includes the following tables: PLAN, MEMBER, ROOM, BOOKING, and ACCESS_LOG.

The primary keys have been defined to uniquely identify each record, and foreign keys enforce relationships between tables. For example:

- MEMBER_PLAN_FK ensures each member references a valid plan.
- BOOKING_MEMBER_FK ensures bookings reference existing members.
- BOOKING_ROOM_FK ensures bookings reference valid rooms.

Using constraints ensures referential integrity, preventing inconsistent data from being inserted.

INSERT SAMPLE DATA

```

INSERT INTO PLAN (plan_id, plan_name, monthly_fee, max_monthly_room_hours)
VALUES ('P1', 'Starter', '50.00', '5.0');

INSERT INTO PLAN (plan_id, plan_name, monthly_fee, max_monthly_room_hours)
VALUES ('P2', 'Pro', '120.00', '15.0');

INSERT INTO MEMBER (member_id, plan_id, full_name, email, phone, status, join_date, PLAN_plan_id)
VALUES ('M1', 'P1', 'Ada Okafor', 'ada@cohub.com', '0700000001', 'ACTIVE', DATE '2026-02-01', 'P1');

INSERT INTO MEMBER (member_id, plan_id, full_name, email, phone, status, join_date, PLAN_plan_id)
VALUES ('M2', 'P2', 'Ben Hughes', 'ben@cohub.com', '0700000002', 'ACTIVE', DATE '2026-02-10', 'P2');

INSERT INTO ROOM (room_id, room_name, room_type, capacity, hourly_rate, is_active)
VALUES ('R1', 'RoomA', 'MEETING', '6', '20.00', 'Y');

INSERT INTO ROOM (room_id, room_name, room_type, capacity, hourly_rate, is_active)
VALUES ('R2', 'Pod1', 'POD', '1', '8.00', 'Y');

INSERT INTO BOOKING (
    booking_id, member_id, room_id,
    start_time, end_time, booking_status, created_at,
    payment_amount, payment_method, payment_status, paid_at,
    ROOM_room_id, MEMBER_member_id
) VALUES (
    'B1', 'M1', 'R1',

```

```

TIMESTAMP '2026-03-02 10:00:00', TIMESTAMP '2026-03-02 12:00:00',
'CONFIRMED', TIMESTAMP '2026-03-01 18:00:00',
'40.00', 'CARD', 'PAID', TIMESTAMP '2026-03-01 18:30:00',
'R1', 'M1'
);

INSERT INTO BOOKING (
  booking_id, member_id, room_id,
  start_time, end_time, booking_status, created_at,
  payment_amount, payment_method, payment_status, paid_at,
  ROOM_room_id, MEMBER_member_id
) VALUES (
  'B2', 'M2', 'R2',
  TIMESTAMP '2026-03-02 09:00:00', TIMESTAMP '2026-03-02 10:00:00',
  'COMPLETED', TIMESTAMP '2026-03-02 08:30:00',
  '8.00', 'BANK', 'PAID', TIMESTAMP '2026-03-02 08:55:00',
  'R2', 'M2'
);

INSERT INTO ACCESS_LOG (
  access_id, member_id, booking_id,
  access_time, access_result, door_location,
  MEMBER_member_id
) VALUES (
  'A1', 'M1', 'B1',
  TIMESTAMP '2026-03-02 09:50:00', 'GRANTED', 'MAIN',
  'M1'
);

INSERT INTO ACCESS_LOG (
  access_id, member_id, booking_id,
  access_time, access_result, door_location,
  MEMBER_member_id
) VALUES (
  'A2', 'M2', 'B2',
  TIMESTAMP '2026-03-02 08:40:00', 'DENIED', 'MAIN',
  'M2'
);

INSERT INTO ACCESS_LOG (
  access_id, member_id, booking_id,
  access_time, access_result, door_location,
  MEMBER_member_id
) VALUES (
  'A3', 'M2', 'B2',
  TIMESTAMP '2026-03-03 23:10:00', 'DENIED', 'SIDE',
  'M2'
);

COMMIT;

```

Sample records have been inserted into each table using INSERT INTO statements.

At least one row has been inserted into each table to demonstrate that:

- the schema works correctly
- the foreign key relationships are valid
- the queries in the next section produce meaningful results

Short identifiers such as P1, M1, R1, B1, and A1 have been used to represent plans, members, rooms, bookings, and access logs.

DISPLAY TABLE CONTENTS

```

340  SELECT * FROM PLAN ORDER BY plan_id;
341  SELECT * FROM MEMBER ORDER BY member_id;
342  SELECT * FROM ROOM ORDER BY room_id;
343  SELECT * FROM BOOKING ORDER BY booking_id;
344  SELECT * FROM ACCESS_LOG ORDER BY access_id;
345

```

This confirms that:

- the data has been inserted successfully
- relationships between tables are valid
- the dataset is ready for querying

PLAN:

	PLAN_ID	PLAN_NAME	MONTHLY_FEE	MAX_MONTHLY_ROOM_HOURS
1	P1	Starter	50.00	5.0
2	P2	Pro	120.00	15.0

MEMBER:

	MEMBER_ID	PLAN_ID	FULL_NAME	EMAIL	PHONE	STATUS	JOIN_DATE	PLAN_PLAN_ID
1	M1	P1	Ada Okafor	ada@cohub.com	0700000001	ACTIVE	2/1/2026, 12:00:00 AM	P1
2	M2	P2	Ben Hughes	ben@cohub.com	0700000002	ACTIVE	2/10/2026, 12:00:00 AM	P2

ROOM:

	ROOM_ID	ROOM_NAME	ROOM_TYPE	CAPACITY	HOURLY_RATE	IS_ACTIVE
1	R1	RoomA	MEETING	6	20.00	Y
2	R2	Pod1	POD	1	8.00	Y

BOOKING:

	BOOKING_ID	MEMBER_ID	ROOM_ID	START_TIME	END_TIME	BOOKING_STATUS	CREATED_AT
1	B1	M1	R1	2026-03-02T10:00:00Z	2026-03-02T12:00:00Z	CONFIRMED	2026-03-01T18:00:00Z
2	B2	M2	R2	2026-03-02T09:00:00Z	2026-03-02T10:00:00Z	COMPLETED	2026-03-02T08:30:00Z

PAYMENT_AMOUNT	PAYMENT_METHOD	PAYMENT_STATUS	PAID_AT	ROOM_ROOM_ID	MEMBER_MEMBER_ID
40.00	CARD	PAID	2026-03-01T18:30:00Z	R1	M1
8.00	BANK	PAID	2026-03-02T08:55:00Z	R2	M2

ACCESS_LOG:

	ACCESS_ID	MEMBER_ID	BOOKING_ID	ACCESS_TIME	ACCESS_RESULT	DOOR_LOCATION	MEMBER_MEMBER_ID
1	A1	M1	B1	2026-03-02T09:50:00Z	GRANTED	MAIN	M1
2	A2	M2	B2	2026-03-02T08:40:00Z	DENIED	MAIN	M2
3	A3	M2	B2	2026-03-03T23:10:00Z	DENIED	SIDE	M2

QUERIES

BOOKING SUMMARY

```
347 SELECT
348     m.full_name,
349     m.email,
350     r.room_name,
351     b.start_time,
352     b.end_time,
353     b.booking_status,
354     b.payment_status,
355     b.payment_amount
356 FROM BOOKING b
357 JOIN MEMBER m ON b.MEMBER_member_id = m.member_id
358 JOIN ROOM r ON b.ROOM_room_id = r.room_id
359 ORDER BY b.start_time;
360
```

	FULL_NAME	EMAIL	ROOM_NAME	START_TIME	END_TIME	BOOKING_STATUS	PAYMENT_STATUS	PAYMENT_AMOUNT
1	Ben Hughes	ben@cohup.com	Pod1	2026-03-02T09:00:00Z	2026-03-02T10:00:00Z	COMPLETED	PAID	8.00
2	Ada Okator	ada@cohup.com	RoomA	2026-03-02T10:00:00Z	2026-03-02T12:00:00Z	CONFIRMED	PAID	40.00

This retrieves a summary of all bookings made by members. It combines information from three tables:

- **BOOKING** – contains the booking records
- **MEMBER** – contains member information
- **ROOM** – contains details about each room

The JOIN operations connect these tables using their foreign key relationships. Specifically:

- BOOKING.MEMBER_member_id links to MEMBER.member_id
- BOOKING.ROOM_room_id links to ROOM.room_id

This returns useful information such as the member name, the room booked, booking times, and payment status.

This type of query would be useful for staff who need to review booking schedules or verify that payments have been completed for reservations.

The results are sorted by booking start time to make the schedule easier to read.

REVENUE PER ROOM

```
361 SELECT
362     r.room_name,
363     COUNT(*) AS paid_bookings,
364     SUM(TO_NUMBER(b.payment_amount)) AS total_revenue
365 FROM BOOKING b
366 JOIN ROOM r ON b.ROOM_room_id = r.room_id
367 WHERE b.payment_status = 'PAID'
368 GROUP BY r.room_name
369 ORDER BY total_revenue DESC;
370
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.017 seconds				
	ROOM_NAME	PAID_BOOKINGS	TOTAL_REVENUE	
1	RoomA	1	40	
2	Pod1	1	8	

This calculates the total revenue generated by each room based on completed payments.

The query joins the BOOKING and ROOM tables so that booking payments can be associated with the rooms where they occurred.

A WHERE condition filters the results to include only bookings with a payment status of PAID, ensuring that pending or failed payments are excluded from revenue calculations.

The GROUP BY clause groups records by room name, allowing the database to calculate revenue per room.

The results have been sorted in descending order of total revenue so that the most profitable rooms appear first.

This helps management understand which rooms generate the highest revenue.

DENIED OR LATE ACCESS ATTEMPTS (SECURITY)

```
371 SELECT
372     m.full_name,
373     a.access_time,
374     a.access_result,
375     a.door_location,
376     a.booking_id
377 FROM ACCESS_LOG a
378 JOIN MEMBER m ON a.MEMBER_member_id = m.member_id
379 WHERE a.access_result = 'DENIED'
380     OR EXTRACT(HOUR FROM a.access_time) >= 22
381 ORDER BY a.access_time DESC;
```

	FULL_NAME	ACCESS_TIME	ACCESS_RESULT	DOOR_LOCATION	BOOKING_ID
1	Ben Hughes	2026-03-03T23:10:00Z	DENIED	SIDE	B2
2	Ben Hughes	2026-03-02T08:40:00Z	DENIED	MAIN	B2

This query analyses access log data to identify potentially suspicious activity.

It retrieves records from the ACCESS_LOG table and joins it with the MEMBER table to show which member attempted access.

The EXTRACT(HOUR FROM access_time) function retrieves the hour component from the timestamp so it can be evaluated, and the system highlights unusual access patterns that may require investigation by security staff.

The results are ordered by time so that the most recent access events appear first.

This query demonstrates how the database can support security monitoring and auditing.

SECURITY, INTEGRITY s ETHICS AND LEGAL CONSIDERATIONS

II.2.a GENERAL DISCUSSION

Modern database systems must ensure that data is protected against unauthorised access, accidental modification, and misuse. Database governance typically focuses on three core aspects: security, integrity, and ethical and legal responsibilities.

Security

Database security focuses on protecting information from unauthorised access, disclosure, or modification. One widely used principle is least privilege, where users are only given the minimum level of access required to perform their tasks. This reduces the risk of accidental or malicious misuse of sensitive data.

Security can be implemented through several mechanisms:

- **Authentication:** verifies the identity of users accessing the system.
- **Authorisation:** controls what actions authenticated users are permitted to perform.
- **Role-based access control (RBAC):** allows permissions to be assigned to roles rather than individual users.

- **Encryption:** protects data both in transit and at rest.
- **Auditing and monitoring:** track database activities to detect suspicious behaviour.

These measures help protect databases from internal threats, external cyberattacks, and accidental data leaks (**Connolly s Begg, 2015**).

Integrity

Data integrity ensures that information stored in a database remains accurate, consistent, and reliable throughout its lifecycle.

Important forms of integrity include:

- **Entity integrity:** which requires every table to have a primary key so that each record is uniquely identifiable.
- **Referential integrity:** which ensures relationships between tables remain valid through foreign key constraints.
- **Domain integrity:** which restricts allowable values within a column using data types or check constraints.

Ethics and legal considerations

Beyond technical controls, organisations must also consider ethical responsibilities when storing and processing personal data. Databases often contain personally identifiable information (PII), such as names, email addresses, and contact details.

Ethical data management involves:

- Collecting only necessary information
- Informing individuals how their data will be used
- Protecting personal data from misuse
- Ensuring data is not retained longer than required

In many regions, data protection laws such as the **General Data Protection Regulation (GDPR)** require organisations to implement strong safeguards for personal information and allow individuals to request access to their data (**European Union, 2016**).

Failure to manage data responsibly can lead to legal penalties, financial losses, and reputational damage.

II.2 .B APPLICATION TO THE COWORKHUB DATABASE SCENARIO

The CoWorkHub database stores several types of sensitive information, including member identities, contact details, booking activity, and building access records. Because of this, appropriate governance mechanisms must be implemented.

Security controls in the CoWorkHub system

Several security practices could be applied to protect this database.

Firstly, role-based access control should be implemented. Different staff members require different levels of access to the system. For example:

- Reception staff may view member and booking information but should not be able to modify payment records.
- Finance staff may update payment information but should not have access to security logs.
- Security staff may view access logs but should not modify booking or payment records.
- Database administrators manage the schema and system configuration.

By assigning privileges to roles rather than individuals, access can be managed more efficiently and securely.

Secondly, authentication mechanisms should ensure that only authorised users can access the database. Password policies and multi-factor authentication could be used to strengthen user verification.

Furthermore, database auditing should be enabled to record important actions such as booking updates, payment changes, or repeated failed login attempts. Audit logs help identify suspicious activity and support incident investigations.

Integrity controls in the database design

The schema designed already includes several mechanisms to preserve data integrity.

Primary keys ensure that each record in the PLAN, MEMBER, ROOM, BOOKING, and ACCESS_LOG tables is uniquely identifiable. Foreign key constraints enforce relationships between entities, such as ensuring that every booking references a valid member and room.

These constraints prevent inconsistent data from entering the system. For example, a booking cannot exist without a corresponding member record, and access log records must reference valid members.

Additional business rules could also be enforced to maintain data quality. For example, the system could prevent overlapping bookings for the same room by validating booking time ranges before inserting new records.

Ethical and legal considerations for CoWorkHub

The CoWorkHub database stores personal information about members, including names, email addresses, and phone numbers. This information must be handled responsibly and in accordance with data protection regulations.

One ethical consideration is data minimisation. Only information necessary for managing memberships and bookings should be collected. Storing unnecessary personal information increases privacy risks.

Another issue involves access log monitoring. While access logs are important for security, they also track individual behaviour within the building. Members should therefore be informed that access attempts are recorded and that the data will only be used for security and operational purposes.

The organisation should also implement data retention policies. For example, access logs may only need to be stored for a limited period before being archived or deleted.

Finally, appropriate safeguards should be implemented to protect stored data. Encryption of sensitive information and secure backup systems can help prevent data loss or unauthorised disclosure.

By combining technical security controls, integrity constraints, and responsible data governance practices, the CoWorkHub database can maintain reliable and ethically managed information systems.

REFERENCES

Connolly, T. C Begg, C., 2015. *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Pearson.

European Union, 2016. *General Data Protection Regulation (GDPR)*. Regulation (EU) 2016/679.